

NOLTE GEOSERVICES

UXO Target Sync Platform

Target Tracker & Data Visualization Platform

Complete Project Documentation

Project: 11-24-2736 — Wilhelmshaven Rüstersieler Seedeich

Repository: C:\Apps\feedbackapp (branch: main)

Generated: 21 August 2026

Table of Contents

- | | |
|--|--|
| 1. What Is This Project? (Plain-Language Overview) | 10. The Data Ingestion Pipeline |
| 2. Key Concepts & Glossary | 11. The Frontend Web Application (React) |
| 3. System Architecture | 12. Offline-First Design & Synchronization |
| 4. Technology Stack | 13. Status Classification Logic |
| 5. Project Folder Structure | 14. How to Run the Project |
| 6. The Database (Data Model) | 15. Configuration Reference |
| 7. The Backend Server (FastAPI) | 16. Troubleshooting |
| 8. API Reference | 17. Known Limitations & Notes for Future Development |
| 9. Coordinate Systems & Conversions | |

About this revision. Reconciled against branch `main` as of 25 September 2026; the previous revision was 21 August 2026. The largest change since then is **how survey data gets into the database**: a configuration-driven ETL pipeline (`etl/`) now loads it from per-project database schemas, and the one-off CSV script, the hand-run SQL migrations, the unreachable import panel and the destructive seed endpoint have all been removed (§10). The platform now carries a second project, Köln Deutzerfeld, and the Wilhelmshaven targets were replaced from new source tables on 24 September 2026.

This document is **internal engineering and handover documentation**, not a client deliverable. §17 is deliberately candid about what is still wrong.

Companion Markdown docs live in the repository and are the maintained day-to-day reference: `README.md` , `docs/ARCHITECTURE.md` , `docs/DATA_PIPELINE.md` , `docs/ETL_DESIGN.md` , `etl/README.md` , `docs/OPERATIONS.md` and `docs/PROJECT_OVERVIEW.md` . Where this PDF and those files disagree, the Markdown is newer.

1. What Is This Project? (Plain-Language Overview)

This project is a field data-collection and visualization platform built for Nolte Geoservices GmbH, a German geophysics company. Its purpose is to support UXO clearance work — the process of finding and safely removing unexploded ordnance (old wartime bombs, mines, and munitions) buried in the ground before construction work begins.

It carries two real projects: “11-24-2736 — Wilhelmshaven Rüstersieler Seedeich”, a sea-dike site in Wilhelmshaven, northern Germany, and “11-26-5151 — Köln Deutzerfeld”. The workflow it supports looks like this:

- 1. Survey phase** (already done, outside this app). Geophysicists scan the site with two instruments — georadar (ground-penetrating radar, GPR) and magnetometers. Each suspicious underground signal is recorded as an *anomaly* (also called a *target*): a point with coordinates, an estimated depth and a category (Kat-1 to Kat-3). The survey results are imported, typically from QGIS, into one database schema per project.
- 2. Loading targets into the platform.** An ETL pipeline (`etl/`) reads those project schemas and adds new targets to the central target table, on a 15-minute schedule. It never deletes a target, and changes to existing ones wait for an approver (\$10). On 25 September 2026 the database held 2,342 targets (2,215 Wilhelmshaven, 127 Köln) and 16 feedback records.
- 3. Field investigation.** A field worker opens the web app on site, sees all targets on an interactive map, walks/digs to each one, and records what was actually found: the excavated depth, dimensions of the hole, the object found (iron bar, wire, nothing, etc.), photos, notes, and the crew and equipment used. This works even without an internet connection — entries are stored in the browser and synced later.
- 4. Office monitoring.** Back at the office, an analyst sees live progress: how many targets are investigated, what was found, charts comparing predicted vs. actual depths, excavated volume, and a map coloured by status. The same surface exports the official *Öffnungsmaßnahmen* report as PDF or CSV.

In one sentence: it is an offline-capable “ArcGIS Field Maps”-style web app plus backend, purpose-built for tracking the investigation of suspected UXO targets and syncing field findings with a central geospatial database.

1.1 Who may see what

Access is granted **per surface**, not per job title. Every account carries three independent flags — `can_field`, `can_dashboard` and `is_admin` — and the server re-checks them on every request. A user who opens a surface they lack is offered a permission request, which an administrator approves or denies in the app. The older `role` value survives only as the landing view and the label shown in the sidebar; it no longer decides anything.

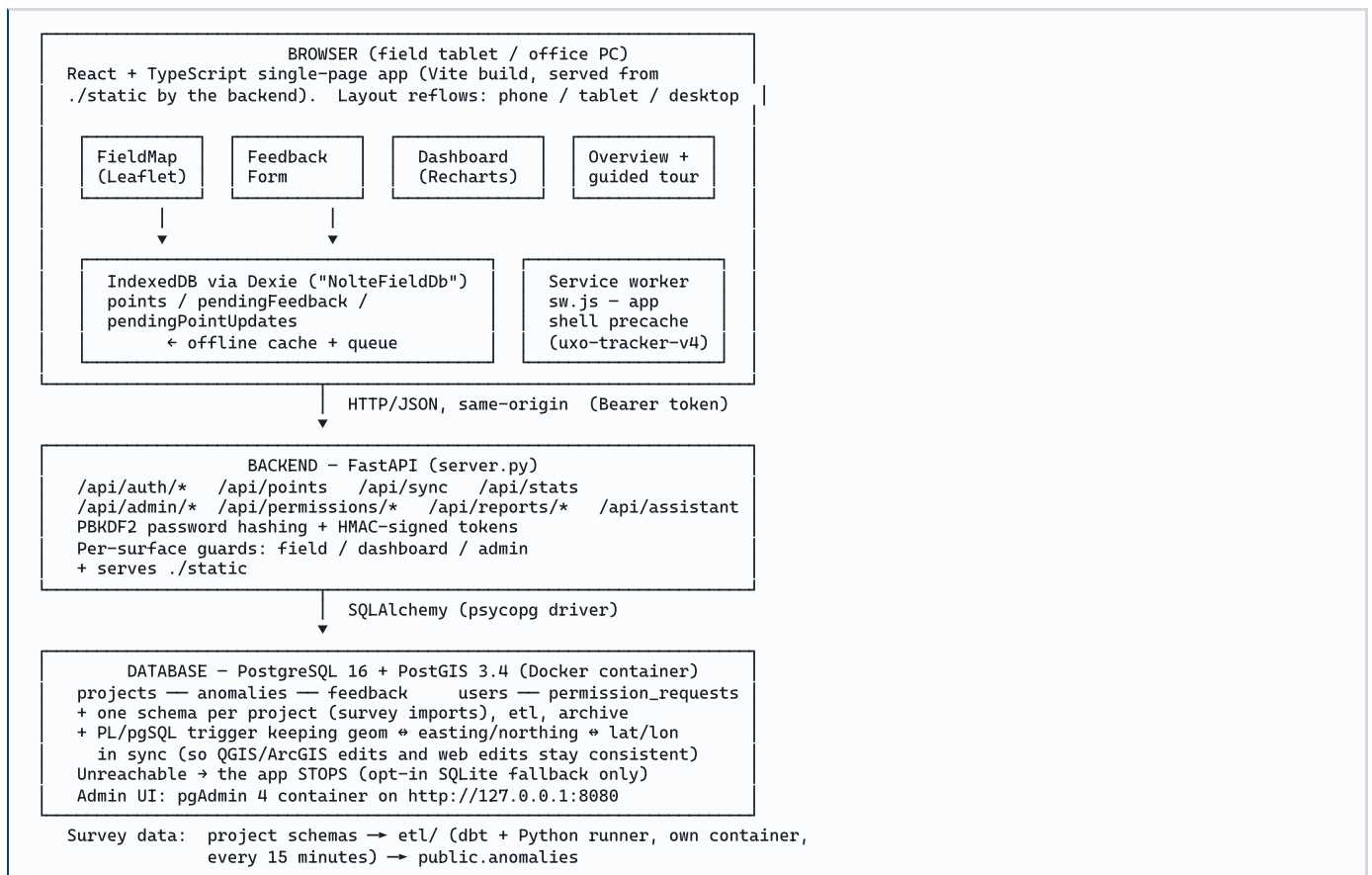
2. Key Concepts & Glossary

Term	Meaning in this project
UXO	Unexploded Ordnance — buried explosive remnants (bombs, mines, shells) that must be located and removed.
Anomaly / Target / Point	A single suspicious spot detected by a survey instrument. The three words are used interchangeably in the code. Stored in the <code>anomalies</code> database table.
Georadar / GPR	Ground-Penetrating Radar — an instrument that images the shallow subsurface with radar pulses.
Magnetic / Magnetometer	An instrument that detects buried ferrous (iron-containing) metal by its magnetic field.
Feedback	The record a field worker creates after physically investigating one target (what was found, how deep, photos, notes, crew). Stored in the <code>feedback</code> table.
VM Nr.	“Verdachtsmoment-Nummer” — a human-friendly sequential target label like <code>2736-17</code> (project short-code + running number).
<code>target_id</code>	A stable, unique string built from project id + coordinates, e.g. <code>11-24-2736-442972.981-5937097.795</code> . Used to derive a deterministic UUID for each anomaly.
Category (Kat-1 ... Kat-3)	The survey's own classification of a target, known before anyone digs. Carried from each project's source data by the ETL pipeline. Not the same as the Fundstück, which is what the excavation found.
Project schema	One database schema per project (e.g. <code>p_11_26_5151_koeln_deutzerfeld</code>) holding the imported survey tables. The ETL pipeline reads them; the app itself never does.
Fundstück (German)	“Found object” — what the excavation uncovered, e.g. Eisenstange (iron bar), Eisendraht (iron wire), <i>ohne Fund</i> (nothing found).

Sohle / Sohle status (German)	"Pit floor." <i>Frei</i> = cleared to the bottom; <i>Nicht Frei</i> = could not be fully cleared (e.g. blocked by a utility pipe).
Tiefe / Länge / Breite / m ³	German for depth / length / width / cubic metres — the dimensions of the excavation opening.
Trupp & Geräte	"Crew and equipment" — the block on the field form recording Truppführer, Maschinenführer, Suchfeld, Messgerät and Sondierer. Stored as JSON in <code>feedback.teams_tools</code> and printed in the report header.
Surface	One of the two application faces: the <i>Field App</i> and the <i>Dashboard</i> . Permissions are granted per surface.
UTM 32N (EPSG:32632)	The metric map-projection coordinate system used by German surveyors (easting/northing in metres). The web map instead needs WGS84 latitude/longitude (EPSG:4326), so the code converts between the two.
PostGIS	A PostgreSQL extension that adds geospatial data types (<code>GEOMETRY</code>) and functions, letting GIS tools like QGIS/ArcGIS read the database directly.
Offline-first	A design where the app keeps working with no network: data lives in the browser (IndexedDB) and is pushed to the server when connectivity returns.

3. System Architecture

The system has three tiers plus a scheduled data pipeline:



Why this shape?

- The field has poor connectivity, so the browser keeps its own full copy of all targets in IndexedDB and queues unsent work. The server is the single source of truth once syncing succeeds.
- The office uses professional GIS tools (QGIS, ArcGIS Pro) that talk to PostGIS directly. Database triggers make sure a point moved in QGIS updates lat/lon for the web app, and a point dragged on the web map updates the PostGIS geometry — bidirectionally.
- One deployable unit: the built React app is written into `./static` and served by FastAPI itself, so a single `python server.py` serves both the API and the UI on port 8000. Because both come from one origin, the frontend calls `/api/...` with an empty base URL and there is no host to configure.
- The asymmetry is deliberate:** the field app is built to survive with no network; the dashboard is not. Analytics and reports are read straight from the server, because a stale dashboard is worse than an unavailable one. The crew's own work is never

blocked that way.

4. Technology Stack

Layer	Technology	Role
Backend	Python 3.13	Language
	FastAPI + Uvicorn	Web framework and ASGI server (port 8000)
	SQLAlchemy 2 (+ psycopg 3)	ORM / database access
	GeoAlchemy2	Maps the PostGIS <code>GEOMETRY</code> column into SQLAlchemy models
	Pydantic + pydantic-settings	Request validation and <code>.env</code> configuration
	hashlib (PBKDF2-HMAC-SHA256)	Password hashing and token signing — standard library only, so there is no crypto dependency to install on a field laptop
Database	PostgreSQL 16 + PostGIS 3.4	Primary store, run via Docker Compose (<code>postgis/postgis:16-3.4</code>)
	pgAdmin 4	Web-based database admin UI (<code>dpape/pgadmin4</code> , port 8080), also in the compose file
	SQLite	Opt-in local fallback (<code>uxo_local.db</code>) for deliberate offline work — no longer automatic , see §7
Frontend	React 19 + TypeScript	UI framework
	Vite 8	Dev server and production bundler (outputs directly to <code>../static</code>)
	Leaflet + react-leaflet 5	Interactive map with switchable basemaps
	Recharts 3	Dashboard charts (bar, pie, area)
	Dexie 4	Friendly wrapper around browser IndexedDB (offline storage)
	lucide-react	Icon set
	PWA assets (<code>manifest.json</code> , <code>sw.js</code>)	Installable app and service-worker offline shell, sourced from <code>frontend/public/</code>
Reporting	reportlab	<code>report.py</code> — the Öffnungsmaßnahmen PDF and the CSV export
Pipeline	dbt (dbt-postgres) + a Python runner	The ETL from the project schemas into <code>public.anomalies</code> (<code>etl/</code>), run as its own Docker Compose service under a least-privilege database role
Landing assistant	Mistral API (<code>ministral-8b-2512</code>)	The “Ask AI Assistant” box on the landing page (<code>assistant.py</code>). Standard-library HTTP, no SDK; no database access
Tooling	Docker Compose, Node.js 22 (fetched by <code>install_node.py</code>), ESLint	Infrastructure and code quality

Dependency pinning. `requirements.txt` is pinned to the versions this app is known to run on. Unpinned, a fresh install resolves whatever is newest that day, which for pydantic or reportlab means a major release landing straight in `Settings` or in PDF generation. To move a pin: bump it, run the app, then commit the version that worked.

5. Project Folder Structure

```
C:\Apps\feedbackapp\
├─ server.py           ← FastAPI backend: auth, API endpoints, reports, static serving
├─ models.py          ← SQLAlchemy tables (Project, Anomaly, Feedback, User, PermissionRequest)
├─ database.py        ← DB connection, retry/fallback, table init, PostGIS trigger
├─ config.py          ← Settings from .env; warns on POSTGRES_PASSWORD / DATABASE_URL mismatch
├─ report.py          ← CSV + Öffnungsmaßnahmen PDF generation (reportlab)
├─ assistant.py       ← The landing page's assistant: POST /api/assistant, Mistral, limits
├─ manage_access.py   ← CLI: list / grant / revoke / set-password
├─ install_node.py    ← Helper that downloads a portable Node.js into .\node
├─ test_utm.py        ← Standalone sanity check for the UTM conversion math
├─ requirements.txt    ← Python dependencies (pinned)
├─ docker-compose.yml ← PostgreSQL/PostGIS + pgAdmin, both bound to 127.0.0.1
├─ .env               ← Live configuration (NOT committed)
├─ .env.example       ← Committed template and the source of truth for every setting
├─ .gitignore         ← Excludes .env, *.db, node/, __pycache__, backup-*.sql, scratch/, archive/
├─ reportTemplate.pdf ← The Excel-exported layout report.py reproduces
├─ scripts\
│   └─ doctor.py      ← Read-only diagnostics for the database, credentials and pgAdmin
├─ etl\
│   └─ config\projects.yml ← Each project's schema, source tables, column mapping, SRID, VM prefix
│       └─ dbt\
│           └─ runner\run.py ← Change check, uuid5 ids, the gated merge, approvals, setup SQL
│               └─ tests\acceptance.py ← Full acceptance suite, run against a copy of the database only
├─ docs\              ← This PDF plus ARCHITECTURE / DATA_PIPELINE / ETL_DESIGN / OPERATIONS / OVERVIEW
├─ static\            ← Production build of the React app (served at "/"), committed
│   └─ index.html, assets\, logo.png, manifest.json, sw.js ...
├─ frontend\          ← React + TypeScript source code
│   └─ package.json, vite.config.ts, tsconfig*.json
│       └─ public\      ← Static assets copied verbatim into the build, incl. sw.js
│           └─ src\
│               └─ App.tsx ← Root component: login, surfaces, sync logic, admin dialogs
│                   └─ auth.ts ← Token storage, access flags, authFetch, offline-login policy
│                       └─ i18n.ts ← EN/DE translation layer
│                           └─ useFilters.ts ← Target filters (project, VM Nr., instrument, category, status)
│                               └─ useIsMobile.ts ← Shared 768px breakpoint for the JS-driven reflow
│                                   └─ geo.ts ← Distance and bearing from the crew to a target
│                                       └─ db\indexedDb.ts ← Dexie schema + status-resolution helper
│                                           └─ tour\
│                                               └─ components\
│                                                   └─ Landing.tsx ← Public landing page, incl. the assistant box
│                                                       └─ AuthDialog.tsx ← Sign-in and sign-up
│                                                           └─ Overview.tsx ← Post-login home: introduction and tour launchers
│                                                               └─ FieldMap.tsx ← Leaflet map, markers, basemaps, resize handling
│                                                                   └─ FeedbackForm.tsx ← Investigation entry form incl. camera capture
│                                                                       └─ Dashboard.tsx ← Analyst dashboard with charts and KPIs
│                                                                           └─ FilterBar.tsx ← VM-Nr. / instrument / status filtering
│                                                                               └─ ReportDialog.tsx ← Project + date range for the CSV/PDF exports
```

The `node\` folder is no longer tracked. The repository used to carry two full portable Node.js distributions — 4,963 of its 5,046 files and 181 MB of the 190 MB. They were untracked and ignored on 20 August 2026, and the superseded v22.11.0 copy was deleted from disk. Recreate the toolchain with `python install_node.py`; nothing else in the repository refers to the directory. Note that this did not shrink `.git` — the blobs remain in history, which would need a rewrite to reclaim.

6. The Database (Data Model)

Five tables, defined in `models.py`. The core chain is one-to-many: a project has many anomalies, and each anomaly can have many feedback records (though in practice the app shows only the latest one). Two further tables carry accounts and access requests.

6.1 projects

Column	Type	Description
<code>project_id</code>	<code>VARCHAR(50)</code> , PK	e.g. 11-24-2736

project_name	VARCHAR(255)	e.g. “Wilhemshaven Rüstersieler Seedeich”
created_at / updated_at	TIMESTAMP	Audit timestamps

6.2 anomalies — the survey targets

Column	Type	Description
id	VARCHAR(36), PK	Deterministic UUIDv5 derived from <code>target_id</code> (see box below)
project_id	FK → projects	Cascade delete
instrument	VARCHAR(50)	georadar or magnetic
geom	PostGIS POINT, SRID 32632	Geospatial geometry for GIS tools
easting / northing	FLOAT	UTM 32N metres (survey coordinates)
latitude / longitude	FLOAT	WGS84 degrees (for the web map)
vm_nr	VARCHAR(50)	Sequential label, e.g. 2736–17
category	VARCHAR(50)	The survey's classification, <code>Kat-1</code> to <code>Kat-3</code> , carried from each project's source by the ETL pipeline
layer	VARCHAR(255)	Source survey layer, e.g. <code>Stoerkoerper Magnetik Nord</code>
status	VARCHAR(50)	pending → investigated
target_id	VARCHAR(100), UNIQUE	{project}-{easting}-{northing}
evaluated_depth	FLOAT	Depth predicted by the survey instrument (m)

Stable IDs — a deliberate design choice. Anomaly IDs are not random. They are `uuid5(NAMESPACE_DNS, target_id)`, i.e. a hash of the project + coordinates. Loading the same target twice always produces the same ID, so feedback recorded against a target never becomes orphaned when the target list is reloaded. (Commit `b39c85b` introduced this.) `models.py` also declares a `uuid4` default on the column, but no insert path reaches it — the ETL pipeline passes an explicit `uuid5`, as every retired loader did. Recomputing the rule for all 1,710 rows stored on 22 September 2026 reproduced every id with zero mismatches, and the pipeline's `dbt test` re-checks it on every run.

6.3 feedback — field investigation results

Column	Type	Description
id	VARCHAR(36), PK	Created client-side so offline entries have an identity
anomaly_id	FK → anomalies	Which target this report belongs to
visited	BOOLEAN	Was the target physically visited
visit_date	TIMESTAMP	When it was logged — also the guard that stops a stale queued copy overwriting a newer visit
tief	FLOAT	Actual excavated depth (m)
laenge / breite / m_cube	FLOAT	Opening length, width (m) and volume (m³)
fundstueck	VARCHAR(255)	What was found (“Eisenteil”, “ohne Fund”, ...)
sohle_status	VARCHAR(50)	<i>Frei / Nicht Frei</i>
bilder_n	INTEGER	Number of photos
photos	TEXT	JSON array of base64-encoded images
notes / other	TEXT	Free-text remarks / “Sonstige” description
investigator / investigator_username	VARCHAR(100)	Who logged it

<code>project_id</code>	FK → projects	Denormalized from the parent anomaly, never taken from the client, so it cannot disagree with the target
<code>target_id</code>	VARCHAR(100)	Denormalized copy for traceability — nothing joins on it
<code>teams_tools</code>	JSON	The Trupp & Geräte block: <code>need_update</code> , <code>truppfuehrer</code> , <code>maschinenfuehrer</code> , <code>bez_suchfeld</code> , <code>messgeraet</code> , <code>sondierer</code> . Travels as one column and drives the report header.

6.4 users — accounts and per-surface access

Column	Type	Description
<code>id</code>	VARCHAR(36), PK	UUID
<code>full_name</code> / <code>username</code> / <code>email</code>	VARCHAR	<code>username</code> is unique and indexed
<code>password_hash</code>	VARCHAR(255)	<code>pbkdf2_sha256\$<iterations>\$<salt>\$<hash></code> — an encoded value is ~110 characters
<code>role</code>	VARCHAR(50)	<code>collector</code> or <code>dashboard</code> . Landing view and sidebar label only — it does not grant access.
<code>can_field</code>	BOOLEAN, default true	May open the Field App
<code>can_dashboard</code>	BOOLEAN, default false	May open the Dashboard
<code>is_admin</code>	BOOLEAN, default false	May administer users and decide permission requests
<code>must_change_password</code>	BOOLEAN, default false	Set when an admin issues a temporary password; blocks every surface until the user chooses their own

6.5 permission_requests

Raised when a user opens a surface they lack, and decided by an administrator. Columns: `user_id` (FK → users), `surface` (`field` or `dashboard`), `status` (`pending` / `approved` / `denied`), `message`, `created_at`, `decided_at`, and `decided_by` (the deciding admin's user id).

6.6 Additive migrations

`create_all` creates missing tables but never alters existing ones, so columns added after a database was first seeded have to be added explicitly. `init_db()` does this by hand for `feedback.teams_tools`, `feedback.project_id` and the four user flags, checking `information_schema` (or `PRAGMA table_info` on SQLite) first. The backfill runs only for columns that particular run added — dashboard-only accounts lose `can_field` and gain `can_dashboard` once — so a later edit to somebody's access is never overwritten on the next startup.

6.7 The spatial synchronization trigger

`database.py` → `init_db()` installs a PL/pgSQL trigger (`sync_anomaly_geom_and_coordinates`) that fires before every insert/update on `anomalies` and keeps all three coordinate representations consistent, whichever one was edited:

- **Case 1 — geom changed** (e.g. a point moved in QGIS/ArcGIS Pro): recompute easting/northing and, via `ST_Transform(..., 4326)`, latitude/longitude.
- **Case 2 — easting/northing changed** (e.g. edited numerically): rebuild `geom` and lat/lon.
- **Case 3 — latitude/longitude changed** (e.g. a marker dragged on the web map): rebuild `geom` in UTM 32N and easting/northing.

Case 2 is why an insert that supplies its own lat/lon but leaves `geom` null will have those values recomputed from SRID 32632 — intentional, so every row obtains its coordinates the same way. When the Köln targets were first appended, the resulting divergence from their source's own lat/lon was measured at most 0.0069 m (see `docs/DATA_PIPELINE.md`).

7. The Backend Server (FastAPI)

`server.py` is the whole API — about 1,000 lines, plus the landing assistant in `assistant.py`. On startup it:

1. Calls `init_db()` : connects to the database, enables PostGIS, creates missing tables, applies the additive column migrations, and installs the sync trigger.
2. Creates the FastAPI app titled "Nolte Geoservices UXO Target Sync Platform".
3. Runs a **lifespan hook** — replacing the deprecated `@app.on_event` handlers — which seeds the default accounts, but **only into an empty users table**.
4. Enables fully-open CORS (any origin/method/header) — convenient for development, still to be tightened for production (§17).
5. Mounts `./static` at `/`, so the compiled React app is served by the same process.

7.1 Connecting to the database: fail loudly

This behaviour was inverted on 19 August 2026 and is the single most important operational change in this revision.

- **The first connection is retried** for `DB_CONNECT_RETRY_SECONDS` (default 30). A cold boot starts this process and the database together, and Postgres needs a few seconds before it accepts connections; nothing supervises this process, so a single attempt would turn an ordinary race into an outage lasting until somebody restarted the app by hand. Set it to `0` to fail on the first attempt.
- **A failed connection is now fatal**. The app previously fell through to SQLite unconditionally, which turned a wrong password or an unreachable server into an app that started cleanly against an empty database: the data was simply absent, every login failed, and the real cause sat in one warning line among the startup output.
- **The SQLite fallback is opt-in**, for deliberate offline work only, via `ALLOW_SQLITE_FALLBACK=1` . See the gotcha in §15 — it is read from the process environment, not from `.env` .
- **The connection URL is masked** in every log line (`safe_url()`). These lines run on every startup into terminal scrollback, log files and CI output; rotating a password and keeping it out of git achieves nothing if the app prints it each time it starts.

`config.py` additionally compares `POSTGRES_PASSWORD` against the password inside `DATABASE_URL` and prints a warning when they disagree — percent-decoding the URL's copy first, so an escaped password does not read as a mismatch. It warns rather than fails, because a volume whose password was rotated with `ALTER USER` sits in that state legitimately.

7.2 Authentication and authorization

Passwords are hashed with PBKDF2-HMAC-SHA256 from the standard library — no external crypto dependency to install on a field laptop — and stored as `pbkdf2_sha256$<iterations>$<salt>$<hash>` . Stored hashes are transparently re-hashed on login when the iteration count moves. Session tokens are HMAC-signed with `AUTH_SECRET` ; if that is unset a random key is generated per process, so every restart signs everyone out.

Authorization is enforced by four FastAPI dependencies:

Dependency	Effect
<code>current_user</code>	A valid bearer token is required.
<code>require_surface("field" "dashboard")</code>	The matching access flag is required. The 403 body carries the surface name, so the client can offer to request that permission rather than showing a bare error.
<code>require_any_surface(...)</code>	At least one of several surfaces. Used for the CSV export, which both apps legitimately offer — requiring <code>dashboard</code> there locked collectors out of a button their own screen shows them.
<code>require_admin</code>	<code>is_admin</code> is required.

Every one of them first blocks a user whose `must_change_password` flag is set. Request and response shapes remain Pydantic models: `UserRegister` , `UserLogin` , `FeedbackCreate` , `PointUpdate` , `SyncPayload` , `TeamsTools` , `UserAccessUpdate` and `PermissionRequestCreate` .

7.3 Default accounts

Seeded only into an empty users table, and given the password in `SEED_PASSWORD` . If that is unset, each account receives its own random password, which has to be reset rather than guessed. Once any account exists, `SEED_PASSWORD` has no effect at all.

Username	Access	Purpose
<code>collector</code>	field	Field App demo account

dashboard	dashboard	Analyst demo account
eric.musonera	field + dashboard + admin	Somebody has to be able to approve the first permission request

8. API Reference

POST `/api/points/import` and POST `/api/seed` were removed on 25 September 2026 (§10). Guards: `OPEN` no token needed · `AUTH` any signed-in user · `FIELD` `DASH` the matching surface flag · `ADMIN` `is_admin`.

8.1 Authentication

Method & Path	Guard	Behaviour
POST <code>/api/auth/register</code>	<code>OPEN</code>	Creates an account; rejects a duplicate username with HTTP 400.
POST <code>/api/auth/login</code>	<code>OPEN</code>	Verifies the password, re-hashes it if the iteration count has moved, and returns a signed token plus the caller's access flags.
POST <code>/api/auth/change-password</code>	<code>AUTH</code>	Sets the caller's own password and clears <code>must_change_password</code> .
GET <code>/api/auth/me</code>	<code>AUTH</code>	Re-reads the caller's own access, so a client that has been offline picks up permissions granted while it was away.

8.2 Access administration

Method & Path	Guard	Behaviour
GET <code>/api/admin/users</code>	<code>ADMIN</code>	Every account with its flags.
PATCH <code>/api/admin/users/{id}/access</code>	<code>ADMIN</code>	Flips <code>can_field</code> / <code>can_dashboard</code> / <code>is_admin</code> . Returns HTTP 409 rather than removing the last administrator — otherwise nobody could grant anything again.
POST <code>/api/admin/users/{id}/reset-password</code>	<code>ADMIN</code>	Issues a temporary password, returns it once , and sets <code>must_change_password</code> . It is never stored in the clear and no endpoint can return it again; the admin reads it off the screen and passes it on.
POST <code>/api/permissions/request</code>	<code>AUTH</code>	Raises a request for a surface the caller lacks, with an optional message.
GET <code>/api/permissions/requests</code>	<code>ADMIN</code>	The decision queue.
POST <code>/api/permissions/requests/{id}/decide</code>	<code>ADMIN</code>	Approves or denies; approval sets the corresponding access flag.

8.3 Targets and synchronization

Method & Path	Guard	Behaviour
GET <code>/api/points</code>	<code>AUTH</code>	Fetches every target with its latest feedback. For each anomaly, finds the newest feedback row (by <code>visit_date</code>), derives a display status (§13), parses the photos JSON, and returns one combined object per target. This is what the frontend caches into IndexedDB. Both surfaces read it.

POST /api/sync	FIELD	Accepts {feedback: [...], point_updates: [...]} . Each feedback item is upserted by primary key with a dialect-specific ON CONFLICT DO UPDATE : a re-send of an unchanged record is a no-op, an edited record overwrites the stored one, and an id already present can no longer fail its batch mates with a duplicate-key error. A WHERE visit_date <= :visit_date guard keeps a stale copy sitting in an offline queue from clobbering a newer visit. Items whose parent anomaly does not exist are skipped with a warning. Incoming ISO-8601 Z timestamps are normalised to naive UTC to match the column type. Marks each anomaly investigated , applies coordinate updates, and responds with counts plus a fresh full point list so the client refreshes its cache in the same round-trip. On commit failure it rolls back and returns HTTP 400.
GET /api/projects	AUTH	Projects that actually carry anomalies, for the report filter dropdown.

8.4 Reports

Method & Path	Guard	Behaviour
GET /api/reports/feedback.pdf	DASH	The Öffnungsmaßnahmen report for an optional project and date range, laid out after reportTemplate.pdf .
GET /api/reports/feedback.csv	FIELD DASH	The same rows as CSV, written with a BOM so Excel renders the umlauts correctly.
GET /api/reports/bilder/{feedback_id}	OPEN	The target of the Bild links inside the PDF: a standalone page with every photo of one VM point and a download link each. Served no-store .

8.5 Analytics, the landing assistant and the web app

Method & Path	Guard	Behaviour
GET /api/stats	DASH	Total/visited/unvisited counts, completion percentage, and a distribution over the statuses clear / scrap / uxo / false_alarm / unvisited .
POST /api/assistant	OPEN	The landing page's "Ask AI Assistant", before sign-in. Touches no user data and no database. Rate-limited (10 questions per IP per 10 minutes, 300 a day in total), and never shows a visitor an error: any failure returns a normal response and the page falls back to its predefined answers. Questions about handling ordnance never reach the model.
GET /	OPEN	Serves static/index.html (the built React app), or a JSON status message if static/ is absent.

9. Coordinate Systems & Conversions

Two coordinate systems coexist, and the project converts between them in these places:

System	Used by	Example
UTM Zone 32N (EPSG:32632) — metres	Surveyors, the project schemas (as ETRS89 / UTM 32N, SRID 25832), PostGIS geometry, excavation reports	E 442 972.981, N 5 937 097.795
WGS84 (EPSG:4326) — degrees	Leaflet web map, GPS	53.57° N, 8.14° E

- **SQL** — the PostGIS trigger uses ST_Transform for every conversion server-side (the authoritative version), and the ETL pipeline uses it to derive lat/lon from each source's own SRID.
- **TypeScript** — latLonToUtm32nJS() in FeedbackForm.tsx converts a dragged marker's new lat/lon back to easting/northing while offline, using the standard Transverse-Mercator series expansion on the WGS84 ellipsoid.
- **Python** — database.py still carries the same formula as latLon_to_utm32n() , which nothing in the app calls. Its inverse and the TypeScript copy of it went with the import panel on 25 September 2026. test_utm.py is a small standalone script for sanity-checking the math.

Duplicated conversion math is a known liability; see §17.

10. The Data Pipeline (etl/)

Survey targets reach `public.anomalies` through one path: a configuration-driven ETL pipeline, built in September 2026 and running on the live database since 24 September 2026. It runs as its own Docker Compose service, `etl`, every 15 minutes (`ETL_INTERVAL_SECONDS=900`), and skips any run where neither the sources, the configuration nor a pending decision has changed.



- **Configuration only.** `etl/config/projects.yml` names each project's schema, source tables, column mapping, SRID and VM prefix. Nothing project-specific is in the code; adding a project means adding an entry there.
- **Never deletes or rebuilds.** New targets are inserted as `pending` with the next VM number never issued before. The `etl_pipeline` role has no `UPDATE`, `DELETE` or `TRUNCATE` on `public.anomalies` at all. A target that no source produces any more is listed and warned about, never removed.
- **Changes to existing targets need an approver.** A changed category, layer, depth or instrument is staged in `etl.change_log`; a new position within `correction_radius_m` of a target that just lost its source is paired, not inserted. A separate login, `etl_approver`, decides through the `etl-approve` service, and the next run carries out exactly what was approved.
- **The identity rule is kept.** `target_id` and the uuid5 id are computed by the runner with Python's `uuid` module, the same call that produced every existing id, and recorded in an append-only registry.
- **Tested every run.** `dbt test` checks keys, relationships, the uuid5 rule, that every feedback row resolves, and that the app's indexes are present. `tests/acceptance.py` runs a full acceptance suite against a *copy* of the database; it refuses the live one.

10.1 The two projects

Project	Sources	Targets
11-24-2736 Wilhelmshaven	<code>Magnetic</code> (<code>magnetic</code> , category from its own column) and <code>Georadar</code> (<code>georadar</code> , category from its own column or, failing that, the trailing number of <code>Target_Pic</code>)	2,215
11-26-5151 Köln	<code>picks</code> (<code>georadar</code>); category <code>'Kat-' field_3</code> , excluding classes 0 and 4. The pipeline also keeps the schema's <code>anomalie_1</code> staging table in step.	127

Counts as of 25 September 2026. Source rows with no category and no usable fallback are left out and listed, never guessed.

10.2 The one-time Wilhelmshaven migration

On 24 September 2026 the Wilhelmshaven targets were replaced by the new `Magnetic` and `Georadar` source tables: 1,430 targets were inserted and 798 old ones with no counterpart in the new sources were removed, taking 52 feedback rows with them by cascade. Both were archived first, in the database's `archive` schema and as a JSON file kept outside git. Feedback logged on a removed target by a device that had not yet synced would have been lost, which is why the migration waited for the devices to sync.

10.3 What it replaced

Removed on 25 September 2026, once the pipeline had replaced them on live; all remain in git history (`cd69945` and `e0aa3a3`):

- `ingest_anomalies.py` and the sample CSVs it read. A destructive first-load tool: it dropped `feedback`, `anomalies` and `projects` and rebuilt them. The database holds all of the CSVs' data.
- **The hand-run `sql/` migrations** that brought in the Köln targets.
- `POST /api/points/import`, `POST /api/seed and ImportExport.tsx`, the panel that called them. No UI path reached any of the three, and `/api/seed` deleted every anomaly and feedback record on one call.

Design, every rule and its reason: `docs/ETL_DESIGN.md`. Operating it: `etl/README.md`.

11. The Frontend Web Application (React)

The UI is a single-page app with two surfaces, both sharing one collapsible sidebar (theme toggle, EN/DE language toggle, navigation, sync button, online/offline indicator, user avatar, sign-out). Which surfaces a user may open is decided by their access flags, not by a role picked at login.

11.1 Login / landing page

Changed since the previous revision. The 16 July 2026 edition of this document stated that “authentication is demo-grade only: any username/password is accepted... There is no server-side auth at all.” **That is no longer true.**

A public landing page with the company brand, a description of the platform, an “Ask AI Assistant” box (§8.5), and a Sign-in dialog with login and sign-up views. Signing in leads to an Overview page that offers a guided tour of each surface the account can open. Authentication is real:

- Credentials are checked **server-side** against a PBKDF2 hash, and a signed token is returned. `auth.ts` attaches it as a bearer token on every API call via `authFetch`.
- The access flags mirrored into `localStorage` decide only what the UI *offers* — the server re-checks them on every request, so a flag flipped in devtools buys nothing.
- Opening a surface the account lacks raises a **permission request dialog** instead of an error; an administrator decides it in the app.
- A user carrying `must_change_password` gets a **non-dismissible** password dialog — the server refuses every surface until it is done.
- Admins additionally get a user-administration dialog (flags, temporary passwords) and the permission-request queue.

Signing in with no network

Nobody can verify a password offline, so `auth.ts` records every account the server *has* authenticated on this device. Offline, only such an account may sign in, and it is granted the **field app only** — the dashboard and admin screens are server-backed and could otherwise display data the person is no longer allowed to see. That record deliberately outlives sign-out: it describes what this device has learned, not who is signed in now, and signing out must not turn the device back into one that will accept an unknown name offline.

11.2 Field App surface

- **Target list panel:** search box, filters by project, VM number, instrument, category and status, and a scrollable card list showing each target's VM number, instrument/layer, finding, and evaluated depth.
- **Map (`FieldMap.tsx`):** a Leaflet map with three switchable basemaps (Esri grey canvas, light or dark with the theme; OpenStreetMap streets; Esri imagery), circle markers coloured by status, popups with the crew's distance and bearing to the target, an auto-zoom controller that fits the filtered points, and a toolbar.
- **Feedback form (`FeedbackForm.tsx`):** opened by selecting a target. Records opening length/width/depth, computed volume, Fundstück (from the standard German list), Sohle status, notes, the Trupp & Geräte block, and photos — either uploaded (client-side compressed) or captured live with the device camera (`getUserMedia` + canvas snapshot). It also supports an edit-location mode where the marker becomes draggable; the new position is converted back to UTM and queued as a point update.
- **Report dialog:** project and date range for the CSV export.

11.3 Dashboard surface

A full-screen map with floating translucent panels (`Dashboard.tsx`) showing KPI tiles (total targets, investigated vs. pending, project count, excavated volume against a 250 m³ target capacity), a Fundstück bar chart, a Sohle *Frei/Nicht Frei* split, and a predicted (`evaluated_depth`) vs. actual (`tief`) depth comparison with a mean depth-error KPI. The same status/instrument filters are wired to the shared map, and the report dialog here also offers the PDF.

11.4 Responsive layout

Field crews bring whatever tablet or phone they own, so all three views — landing page, field app and dashboard — now reflow. Three tiers:

Tier	Breakpoint	Behaviour
------	------------	-----------

Desktop	> 1024px	The original layout: sidebar, side-by-side panels, dashboard panels floating over a full-bleed map.
Tablet	769–1024px	Intermediate tier — narrower panels and reduced chrome, rather than jumping straight from desktop to phone.
Phone	≤ 768px	Everything reflows into a single scrolling column.

Three implementation notes worth keeping in mind before editing the layout:

- **One breakpoint, two languages.** `useIsMobile.ts` exports `MOBILE_BREAKPOINT = 768` and the CSS overrides key off the same number, so a layout can never be half-switched. The hook exists only because the dashboard's desktop layout floats absolutely-positioned panels over the map: the map is not a sibling in the flow, so no media query can reorder it into a reading column. It uses `useSyncExternalStore` over a single shared `matchMedia` object, so the first render already sees the correct breakpoint instead of painting the desktop layout and correcting it.
- **The landing page needed inline styles overridden.** Its layout lived in inline `style` attributes, which beat any stylesheet rule at any specificity — which is why `landing.css` had no layout media queries at all. At 375px the showcase column was laid out fully outside the viewport and then clipped by `overflow-x: hidden`, so the carousel was missing rather than merely squeezed.
- **Leaflet has to be told its container changed.** Leaflet measures its container once, at creation, and never notices it changing; tiles for the area outside the original box are never requested, which is what the grey panels were — the map was not broken, it did not know it had grown. `invalidateSize` was called nowhere in the codebase. A `ResizeObserver` on the container now catches the phone reflow, a drawer opening beside the map and a panel stacking below it; `orientationchange` is listened for separately because on a phone it can arrive before the box has settled. Calls are coalesced to one per animation frame, since a rotation fires the listeners repeatedly and `invalidateSize` forces a synchronous re-layout every time.

Do not remove the `cssTarget` pin in `vite.config.ts`. Left unpinned, esbuild minifies media queries into range syntax — `@media (max-width: 768px)` becomes `@media (width≤768px)` — which Chrome <104, Safari <16.4 and Firefox <102 do not parse. Those browsers discard the whole at-rule, which silently takes every mobile rule in `index.css` with it and leaves a phone rendering the desktop layout. The landing page survives it because it is fluid by default; the dashboard and field app do not.

11.5 Small fixes worth recording

- **Avatar initials.** All three call sites built initials by slicing the first two characters off one string, so “Emmy Blaize” rendered as *EM* instead of *EB*, and the seeded “Operations Analyst” as *OP*. It only ever looked right for people whose first two letters happened to match their initials. A single `initialsFor()` helper now takes first initial + last initial for a full name and keeps the first-two-letters behaviour for a single token, which the username fallback relies on for values like `admin`. Whitespace runs are collapsed before splitting, and the string is walked by code point so a name starting outside the BMP is not cut in half.
- **EN/DE translation.** `i18n.ts` is a lightweight lookup where the English string is the key: `t('Field App')` returns the German text in DE mode and falls back to the key itself when no translation exists.

12. Offline-First Design & Synchronization

This is the heart of the app. Two **independent** mechanisms are involved, and they are easy to confuse: the service worker caches the application shell so the app *opens* offline; Dexie caches the data so it is *usable* offline.

12.1 The browser database (Dexie / IndexedDB)

`indexedDb.ts`, database name `NolteFieldDb`, version 2, three tables:

Table	Contents
<code>points</code>	The full local mirror of all targets (+ their latest feedback), refreshed from the server whenever online.
<code>pendingFeedback</code>	Outbound queue: feedback entries created while offline, or not yet accepted by the server.
<code>pendingPointUpdates</code>	Outbound queue: coordinate corrections from dragged markers.

Reads always come from Dexie, never directly from the API, so the UI has no online path to special-case.

12.2 The synchronization lifecycle

1. **On login (online):** `fetchFromServer()` pulls `GET /api/points` and rewrites the local `points` table.
2. **Recording feedback:** `handleSaveFeedback()` always writes to the pending queues and optimistically updates the local point — the marker turns green immediately, network or not.
3. **Connectivity monitoring:** the app listens to the browser's `online / offline` events and shows a toast plus an indicator in the sidebar.
4. **Auto-sync:** whenever the app is online, logged in, and the pending count > 0 , `handleSync()` fires automatically (also available as a manual sidebar button with a badge showing the queue size). Background runs are **silent** — the crew did not ask for them, and a failure is harmless because the records stay queued.
5. **Snapshot semantics:** the run captures the ids it is sending and, on success, deletes **only those**. A record queued while the request was in flight survives for the next cycle, and an accepted record is never re-sent.
6. **Coalescing:** saving a record starts a sync *and* bumps the counter the auto-sync effect watches, so two cycles used to POST the same row concurrently and the loser came back as a duplicate-key HTTP 400. A run now in flight absorbs whatever was queued behind it.
7. **Conflict handling:** the server upserts per feedback id and refuses to let an older `visit_date` overwrite a newer one, and silently skips feedback whose target no longer exists (logging a warning).

12.3 The service worker (app shell)

`frontend/public/sw.js`, cache `uxo-tracker-v4`. Registered from `main.tsx` on window load.

- **Install** precaches `/`, `/index.html`, `/manifest.json` and `/favicon.svg`. The typeface is now self-hosted in the bundle, so it is cached on first use like the rest of the build.
- **Activate** deletes every cache whose name is not `CACHE_NAME`, and claims open clients. Renaming the cache is therefore how a bad one is evicted — bump it whenever the precache list changes.
- **Fetch:** `/api/` always goes to the network and is never cached. The document itself (`/`, `/index.html`, any navigation) is network-first, falling back to the cache offline: Vite names every asset by content hash, so a new build is only reachable through the new `index.html`, and serving that from cache first pinned devices to an old build. Everything else is stale-while-revalidate — a cached response is served immediately and refreshed in the background; a miss goes to the network and is cached if it is a basic 200, which is what captures Vite's hash-named JS and CSS. A failed navigation falls back to `/index.html`.

Why offline mode was broken, and what fixed it (19 August 2026). `cache.addAll()` is atomic: a single failed request rejects the whole call, which rejects the `waitUntil` promise, which fails the install event, so the worker never activates and **nothing at all is cached**. The precache list carried a stale `/favicon.ico` — inherited boilerplate; there has never been a `.ico` in this repository, `index.html` has always shipped an SVG favicon. That one 404 was silently costing the app its entire offline mode while `register()` still resolved and logged success in the page.

The fix removes the stale entry and switches to caching each asset individually with `Promise.allSettled`, logging `Precache skipped:` for any that fail. One unreachable URL now costs only that asset. This matters beyond the favicon: the font stylesheet is cross-origin, so a crew installing the app somewhere with no signal would otherwise end up with no offline shell at all — the exact situation this app exists for. The cache was renamed from `v1` to `v2` so the activate handler drops the empty cache already sitting on installed devices. (It has since moved to `v4`, when the font stylesheet left the precache.)

Map tiles are third-party and are **not** precached: only tiles already fetched while online remain available. Target markers come from IndexedDB and work regardless.

13. Status Classification Logic

A target's display status is **derived, not stored**. The same rule set exists in Python (`GET /api/points`) and TypeScript (`getResolvedStatus()`) so an offline device classifies its own records identically. Evaluated top-down on the latest feedback:

Order	Rule	Status	Meaning / colour
0	No feedback, or not visited	<code>unvisited</code>	Red marker — still to do
1	Fundstück = "ohne Fund" (nothing found)	<code>false_alarm</code>	The instrument signal had no physical cause

2	Notes/other text contains “uxo”, “mine”, “bomb”, “munition”, or “pmn”	uxo	Live ordnance — highest priority
3	Sohle status = “Nicht Frei”	scrap	Something found, pit not fully cleared
4	Otherwise	clear	Investigated and resolved (green marker)

Rule 2 is a keyword search over free text and is fragile; see §17.

14. How to Run the Project

Prerequisites

- Python 3.13 with pip (the repository uses a virtualenv at `.venv`)
- Docker Desktop, for PostgreSQL/PostGIS and pgAdmin. **No longer optional** — the app refuses to start without a reachable database unless you deliberately opt into the SQLite fallback.
- Node.js 22 — only needed to rebuild the frontend, since `static/` is committed. Run `python install_node.py` to fetch a portable copy into `.\node`.

Step 1 — Configure

Copy the committed template and fill it in:

```
copy .env.example .env
```

`.env.example` is the source of truth for every setting. It carries the explanation next to each key, and this document deliberately does not restate values. `.env` is gitignored and is read by two different consumers: pydantic-settings (the application) and docker-compose (the containers). Read §15 before choosing passwords — there are three of them and they are not interchangeable.

Step 2 — Start the database (and pgAdmin)

```
cd C:\Apps\feedbackapp
docker compose up -d
# starts two containers, both bound to loopback only:
# feedback_postgres_db → PostgreSQL 16 + PostGIS 3.4 on 127.0.0.1:5432
# nolte-pgadmin → pgAdmin 4 web UI on http://127.0.0.1:8080
```

Credentials for both come from `.env`; compose uses the `${VAR:?message}` form, so a missing value fails the command with that message instead of quietly falling back to a well-known default. Inside pgAdmin, register a server with host `db` (the compose service name), port 5432, and the user and password from your `DATABASE_URL`.

Step 3 — Install Python dependencies and start the backend

```
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python server.py # serves API + web app on http://localhost:8000
```

Step 4 — Load survey data with the ETL pipeline

Survey data is imported into its project schema first, then loaded by the pipeline. The one-time setup creates its two database logins and grants (see `etl/README.md`); after that:

```
docker compose --profile etl run --rm etl run # one run, now
docker compose --profile etl up -d etl # the 15-minute schedule (ETL_INTERVAL_SECONDS)
docker compose --profile etl run --rm etl-approve # anything waiting for a decision
```

Step 5 — Open the app

Browse to `http://localhost:8000` and sign in. On a database seeded from empty, the accounts are `collector`, `dashboard` and `eric.musonera` with the password from `SEED_PASSWORD` — or with individual random passwords if that was unset, in which case reset one with:

```
.venv\Scripts\python manage_access.py set-password eric.musonera
```

Step 6 — Check the setup is sane (optional but recommended)

```
.venv\Scripts\python scripts\doctor.py
```

Read-only diagnostics — see §16.

Frontend development mode (optional)

```
cd frontend
npm install
npm run dev      # Vite dev server with hot reload, proxies /api to :8000
npm run build    # production build, written directly into ../static
```

The dev server binds `host: true`, so a tablet on the same network can reach it for real-device testing. `vite build` uses `emptyOutDir`: everything in `static\` is deleted and rewritten, so nothing the build does not produce may live there. Because `static\` is committed, keep asset rebuilds in their own commit.

15. Configuration Reference

All settings live in `.env`, which is gitignored. The committed `.env.example` documents every key and is the source of truth; no live values are reproduced here.

Setting	Read by	Notes
DATABASE_URL	the app, <code>doctor.py</code>	The password inside this URL is the one that actually connects. <code>postgresql://</code> and <code>postgresql+psycpg2://</code> prefixes are auto-rewritten to the <code>psycpg 3</code> driver. Unreachable → the app stops (§7.1).
AUTH_SECRET	the app	Signs session tokens. Unset → a random key per process, so every restart signs everyone out. Changing it does the same, once. Generate with <code>python -c "import secrets; print(secrets.token_hex(32))"</code> .
SEED_PASSWORD	the app	Given to the default accounts, and only when the users table is seeded empty. No effect once any account exists.
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	docker-compose → Postgres	<code>POSTGRES_PASSWORD</code> is applied only when the <code>postgres_data</code> volume is first created . On an existing volume it is inert; rotate with <code>ALTER USER</code> and update <code>DATABASE_URL</code> to match.
PGADMIN_DEFAULT_EMAIL / PGADMIN_DEFAULT_PASSWORD	docker-compose → pgAdmin	Applied only when the <code>pgadmin_data</code> volume is first created. On an existing volume, change it from inside pgAdmin.
ETL_DB_NAME / ETL_DB_USER / ETL_DB_PASSWORD	docker-compose → the <code>etl</code> service	The pipeline's own least-privilege login (<code>etl_pipeline</code>). Created by the one-time setup SQL.
ETL_APPROVER_USER / ETL_APPROVER_PASSWORD	docker-compose → <code>etl-</code> <code>approve</code>	The separate login that approves or rejects staged changes.
ETL_INTERVAL_SECONDS	docker-compose → the <code>etl</code> service	The schedule. <code>0</code> (or unset) runs nothing on its own; live uses <code>900</code> .
MISTRAL_API_KEY	<code>assistant.py</code> , process environment only	The landing assistant's key. Unset → the page shows its predefined answers.
ALLOW_SQLITE_FALLBACK	<code>database.py</code> , process environment only	<code>1</code> permits the SQLite fallback for deliberate offline work. See the gotcha below.
DB_CONNECT_RETRY_SECONDS	<code>database.py</code> , process environment only	How long to retry the first connection. Default <code>30</code> ; <code>0</code> fails on the first attempt.
Host / port	<code>config.py</code>	<code>0.0.0.0:8000</code> , also hard-coded in <code>server.py</code> 's <code>__main__</code> block.
API_BASE	<code>frontend/src/App.tsx</code>	Now the empty string — the frontend is served by the backend, so every call is same-origin and there is no host to configure. (It was hard-coded to <code>http://localhost:8000</code> in the previous revision.)

15.1 The three passwords

Three different things that happen to look alike, and confusing them has cost real time:

Password	Consumer	Effective when
POSTGRES_PASSWORD	Postgres <code>initdb</code> , via <code>compose</code>	First creation of the <code>postgres_data</code> volume only. Inert thereafter.
the password inside <code>DATABASE_URL</code>	the application	Every connection. This is the one that makes logins work.
PGADMIN_DEFAULT_PASSWORD	pgAdmin, via <code>compose</code>	First creation of the <code>pgadmin_data</code> volume only.

`config.py` warns when the first two disagree. It warns rather than fails, because a rotated volume legitimately sits in that state — but it still matters, because a *recreated* volume would be initialised with `POSTGRES_PASSWORD` and the app would then be unable to connect with the `DATABASE_URL` it has.

`ALLOW_SQLITE_FALLBACK` is not read from `.env`. `database.py` reads it with `os.getenv`, and **nothing in this project loads `.env` into `os.environ`** — the application reads that file only through `pydantic-settings`. Putting the variable in `.env` therefore has no effect whatsoever. Set it in the shell instead:

```
$env:ALLOW_SQLITE_FALLBACK = "1"
```

The same is true of `DB_CONNECT_RETRY_SECONDS` and `MISTRAL_API_KEY`.

15.2 pg_hba and trust authentication

`initdb` writes `trust` for local and `127.0.0.1` connections by default, meaning any process inside the container reaches the database as superuser with no password. The `compose` file therefore sets `POSTGRES_INITDB_ARGS`: `"--auth-local=scram-sha-256 --auth-host=scram-sha-256"`, so a freshly created volume comes up requiring a password. The existing running volume was corrected in place — that setting cannot retroactively change a volume that already exists. `doctor.py` re-checks this on every run.

16. Troubleshooting

Run this first. `.venv\Scripts\python scripts\doctor.py` — whenever the app will not start, logins fail, accounts appear to be missing, or pgAdmin refuses you. It codifies the checks that were previously done by hand, one shell command at a time, the last time nobody could log in; that session turned up three separate causes at once — a stale `POSTGRES_PASSWORD`, a pgAdmin password that had never been applied, and `pg_hba` still on `trust` — none of which announced themselves.

It is read-only: it connects, reads and reports, and changes nothing. It prints no password; where one must be identified it is named by origin ("`DATABASE_URL`"), never by value. pgAdmin is inspected by reading its database, never by attempting a login, which would count against the lockout threshold and could lock the very account being diagnosed. Exit status is 1 if any check fails, so it can gate a deploy or run in CI.

Check	Catches
<code>POSTGRES_PASSWORD</code> vs <code>DATABASE_URL</code>	A rotation applied to only one of them.
Which credential Postgres accepts	A wrong password, distinguished from nothing listening on the port at all.
App startup target	Silently running on the SQLite fallback instead of PostgreSQL.
<code>pg_hba</code> trust rules	A volume still authenticating without a password.
pgAdmin account state	Locked accounts and accumulated failed attempts.

Logins fail, or the accounts are gone

Almost always the database connection, not the accounts. In order of likelihood: the app is running against the **SQLite fallback** — an empty database with no users, which `doctor.py` reports explicitly; `DATABASE_URL` carries a password Postgres does not

accept; `AUTH_SECRET` is unset or was changed, so existing sessions no longer verify; or the account genuinely lacks that surface. To get back in without the web app:

```
.venv\Scripts\python manage_access.py list
.venv\Scripts\python manage_access.py grant <username> --admin --dashboard
.venv\Scripts\python manage_access.py set-password <username>
```

The CLI refuses to revoke the last administrator, needs an interactive terminal for `set-password` (it will say so rather than hang on a pipe), strips the trailing carriage return MinTTY can hand back — hashing that would store a password nobody could ever type again — and re-reads the row afterwards to verify the new password against the stored hash.

The app refuses to start: “Cannot reach the database at ...”

Working as intended (§7.1). Fix the connection; run `doctor.py`. Only set `ALLOW_SQLITE_FALLBACK=1` for deliberate offline work, and remember it is read from the process environment. If the app instead pauses at startup logging “Database not ready (attempt N)”, that is the bounded retry window — it buys time for a slow database, not for a wrong one.

“database files are incompatible with server” on docker compose up

```
FATAL: database files are incompatible with server
DETAIL: The data directory was initialized by PostgreSQL version 16,
       which is not compatible with this version 15.x
```

Cause: the Docker volume keeps the database files between runs, stamped with the PostgreSQL major version that created them. If the `image:` tag points at an older major version than the volume's `PG_VERSION`, PostgreSQL refuses to start.

Fix (non-destructive, preferred): set the image to the major version matching the data — currently `postgis/postgis:16-3.4` — and `docker compose up -d` again. Check what the volume holds with:

```
docker run --rm -v feedbackapp_postgres_data:/data alpine cat /data/Pg_VERSION
```

Fix (destructive): if the data is disposable, `docker compose down -v` and let the pinned image re-initialize — this erases both the database and pgAdmin volumes.

Sync fails with HTTP 400 “database has not been seeded with targets yet”

`POST /api/sync` returns this when the commit fails — most commonly because feedback references anomalies that do not exist in the current database (you are on the SQLite fallback, or the database is new and the pipeline has not loaded it yet). Load targets first, then sync again; queued items are preserved in the browser until a sync succeeds. Check the server log for `Skipping sync of feedback log ...` lines, which name the orphaned records.

Map shows no points after login

Usual causes, in order: the backend is not running; the database is empty (run the ETL pipeline, §14 Step 4); or active filters exclude everything — reset them to “All”.

New survey data does not appear

Check that the `etl` container is running (`docker ps`). Docker restarts it after a crash or a Docker restart, but not after a deliberate stop or `docker kill`, and nothing comes back after a Windows reboot until Docker Desktop is opened. Restart it with `docker compose --profile etl up -d etl`. If it is running, look for staged changes waiting for the approver (`docker compose --profile etl run --rm etl-approve`) and for source rows it left out for lack of a category.

Offline mode is not working

Open DevTools → Application → Service Workers; confirm the worker is activated and look for `Precache skipped:` warnings in the console. Remember that renaming `CACHE_NAME` is what evicts a bad cache (§12.3), and that map tiles are never precached. A hard reload with “Update on reload” checked is the fastest way to get a clean worker.

Mobile layout looks like the desktop one

Check that the `cssTarget` pin in `vite.config.ts` is still there (§11.4).

Taking a backup

The database holds the `users` table, so dumps are sensitive; `.gitignore` excludes `backup-*.sql` for that reason, so keep that naming. Take one before anything destructive:

```
docker exec feedback_postgres_db pg_dump -U postgres nolte_geoservices > backup-nolte_geoservices-<timestamp>
```

17. Known Limitations & Notes for Future Development

This section is deliberately blunt. It is a handover document; the candour is the point.

17.1 Resolved since earlier revisions

Was	Now
"No real authentication. Login is client-side theatre: passwords are never checked, roles live in localStorage, and every API endpoint is unauthenticated."	Server-side PBKDF2 auth, signed tokens, per-surface access flags, forced password change, an admin-decided permission-request flow, and a deliberate offline sign-in policy (§7.2, §11.1).
"Destructive endpoints are exposed: anyone who can reach the server can call <code>POST /api/seed</code> and wipe all data."	Admin-gated in August 2026, then removed on 25 September 2026 with <code>POST /api/points/import</code> and the unreachable import panel (§10.3).
One hard-coded project, loaded by a destructive one-off CSV script that needed an unpinned pandas and CSVs at the repository root.	A configuration-driven ETL pipeline that never deletes, carries two projects and runs on a schedule (§10). The script, the CSVs and the <code>sql/</code> migrations are gone.
"A <code>.gitignore</code> would help. The <code>node/</code> folder (two full Node distributions) makes the repo heavy."	Both done. <code>.gitignore</code> covers <code>.env</code> , <code>*.db</code> , <code>__pycache__</code> , <code>backup-*.sql</code> , <code>scratch/</code> and a root-anchored <code>/node/</code> ; the 181 MB of vendored Node was untracked and the superseded copy deleted. Note the blobs remain in <code>.git</code> history.
Silent SQLite fallback masking an unreachable database.	Fatal by default, opt-in via <code>ALLOW_SQLITE_FALLBACK</code> , with a bounded connect retry, a masked URL in logs, a credential-mismatch warning, and <code>scripts/doctor.py</code> (§7.1, §16).
Desktop-only layout.	Three responsive tiers across landing, field app and dashboard, plus the Leaflet <code>invalidateSize</code> handling and the <code>cssTarget</code> pin (§11.4).
Offline mode silently broken — the service worker never completed installation, so nothing was cached.	Fixed: stale favicon entry removed, per-asset caching via <code>allSettled</code> , cache renamed to <code>uxo-tracker-v2</code> (§12.3).

17.2 Still live — worth fixing

- **Open CORS.** `allow_origins=["*"]` with `allow_credentials=True` — convenient in development, wrong anywhere else. Any page on any origin can call this API; the only thing standing in the way is possession of a bearer token.
- **N+1 query pattern.** `GET /api/points` runs one feedback query per anomaly — fine for hundreds of targets, slow at the 2,342 now loaded and growing. Worse than it first looks, because `/api/stats` and `/api/sync` both call it internally, so every sync round-trip pays the cost. A join or eager-load would fix it.
- **Photos as base64 in the database** will bloat rows quickly; object storage with references and thumbnails would scale better. They also travel inside the sync payload, which is the slowest part of a field upload.

- **Keyword-based UXO detection.** Searching notes for “mine”, “bomb”, ... is fragile — a note reading “nothing resembling a mine” classifies as UXO. An explicit status chosen by the investigator would be more reliable, and this is the one classification where a false negative is a safety matter.
- **Feedback for a target that no longer exists is silently lost.** `/api/sync` skips it with only a server log warning and still answers success, so the device drops it from its queue. Concrete since the Wilhelmshaven migration removed 798 targets. The server should reject it per record and the device keep it queued.
- **An idle device never refreshes its target list.** It downloads the list only at sign-in, on the Sync button, or when records are queued, so new or removed targets reach an open, online device only when someone presses Sync.
- **Pending ETL approvals are invisible in the app.** They wait for the `etl_approver` login and show only in the runner's log and the command-line status view. An admin screen, or at least a badge, would stop decisions from waiting unnoticed.
- **feedback → anomalies is ON DELETE CASCADE.** Deleting a target silently deletes its excavation record; `RESTRICT` would make an accidental delete fail loudly.
- **Duplicated conversion math** in SQL, TypeScript and an unused Python copy. The TS/Python versions also ignore the small geoid differences a library like `pyproj/proj4js` would handle.
- **No automated tests for the app and no CI.** The ETL pipeline has `dbt test` on every run and an acceptance suite; the application has only ad-hoc UTM conversion scripts (`test_utm.py`, `scratch/`). `doctor.py` is CI-ready and would be a reasonable first gate.
- **PostGIS extension errors are only logged.** The enable step in `init_db()` catches and logs on failure, so a failed PostGIS setup continues silently into a broken trigger installation.
- **Deployment.** Everything runs locally — unicorn plus Docker, no HTTPS, no process supervision, and no backup or retention policy for the Postgres volume.

17.3 Setup gotchas — the things that actually cost time

- **Three unrelated passwords** (§15.1). `POSTGRES_PASSWORD` is inert on an existing volume; the password inside `DATABASE_URL` is the one that connects; pgAdmin's is a third thing entirely.
- **ALLOW_SQLITE_FALLBACK is read from the process environment, not .env** (§15.1). Putting it in `.env` does nothing at all.
- **The pipeline stays down after a deliberate stop.** The `etl` service's `unless-stopped` policy restarts it after a crash, but not after `docker stop` or `docker kill`, and nothing in Docker returns after a Windows reboot until Docker Desktop is opened (§16).
- **install_node.py hardcodes its target directory** as `c:\Apps\feedbackapp\node`. On any other machine, edit `TARGET_DIR` or install Node yourself. *Known gap; to be made relative to the script.*
- **static/ is committed build output.** Frontend changes are only live after `npm run build`, and that rebuild belongs in its own commit.
- **SEED_PASSWORD only applies to an empty users table.** After that it is ignored, which reads as “it did nothing”.